

Added Features

How I would make changes to build on my assignment

Change 1: Add dueDate Field

Request: *"Add a `dueDate` field to tasks and display it on the card"*

Files Modified: `task.ts` → `TaskCard.tsx` → `TaskList.tsx` → `sampleTasks.ts`

dueDate — Step 1: Types

```
// src/types/task.ts
export interface Task {
  id: string;
  title: string;
  description: string;
  status: TaskStatus;
  priority: TaskPriority;
  dueDate?: string; // <-- Add optional field
}
```

dueDate — Step 2: TaskCard Props

```
// src/components/TaskCard/TaskCard.tsx
interface TaskCardProps {
  title: string;
  description: string;
  status: TaskStatus;
  priority: TaskPriority;
  dueDate?: string; // <-- Add here too
}
```

dueDate — Step 3: Display Logic

```
// src/components/TaskCard/TaskCard.tsx
// Add below description, before footer
{dueDate && (
  <p className="task-card-due-date" style={{ color: colors.textSecondary }}>
    Due: {new Date(dueDate).toLocaleDateString()}
  </p>
)}
```

- **Conditional render** — Only renders if `dueDate` exists
- **Date formatting** — `toLocaleDateString()` for readable output

dueDate — Step 4: Pass Prop in TaskList

```
// src/components/TaskList/TaskList.tsx
<TaskCard
  title={task.title}
  description={task.description}
  status={task.status}
  priority={task.priority}
  dueDate={task.dueDate} // <-- Add this line
/>
```

dueDate — Step 5: Sample Data

```
// src/data/sampleTasks.ts
{
  id: '1',
  title: 'Design System Setup',
  description: '...',
  status: 'done',
  priority: 'high',
  dueDate: '2025-01-15', // <-- Add to some tasks
},
```

dueDate — Verification

- Card displays "Due: 1/14/2025" format
- Cards without dueDate show nothing (no errors)
- TypeScript catches any missing prop updates

Change 2: Add Search Filter

Request: *"Add a search box to filter tasks by title"*

Files Modified: `useTaskFilter.ts` → `TaskList.tsx` → `TaskList.css`

Search — Step 1: Add State

```
// src/hooks/useTaskFilter.ts
const [filterStatus, setFilterStatus] = useState<FilterStatus>('all');
const [filterPriority, setFilterPriority] = useState<FilterPriority>('all');
const [searchQuery, setSearchQuery] = useState<string>(''); // <-- NEW
```

Search — Step 2: Update Interface

```
// src/hooks/useTaskFilter.ts
interface UseTaskFilterResult {
  // ... existing properties
  searchQuery: string;
  setSearchQuery: (query: string) => void;
}
```

Search — Step 3: Filter Logic

```
// src/hooks/useTaskFilter.ts
const filteredTasks = useMemo(() => {
  return tasks.filter((task) => {
    const matchesStatus = filterStatus === 'all' || task.status === filterStatus;
    const matchesPriority = filterPriority === 'all' || task.priority === filterPriority;
    const matchesSearch = searchQuery === '' ||
      task.title.toLowerCase().includes(searchQuery.toLowerCase());
    return matchesStatus && matchesPriority && matchesSearch;
  });
}, [tasks, filterStatus, filterPriority, searchQuery]);
```

Search — Step 4: Return & Destructure

```
// src/hooks/useTaskFilter.ts – add to return
return { ...existing, searchQuery, setSearchQuery };

// src/components/TaskList/TaskList.tsx – destructure
const { searchQuery, setSearchQuery, ...rest } = useTaskFilter(tasks);
```

Search — Step 5: Input JSX

```
// src/components/TaskList/TaskList.tsx
<div className="task-list-search">
  <input
    type="text"
    placeholder="Search tasks..."
    value={searchQuery}
    onChange={(e) => setSearchQuery(e.target.value)}
    className="search-input"
    style={{
      backgroundColor: colors.surface,
      color: colors.text,
      borderColor: colors.border,
    }}
  />
</div>
```

Search — Verification

- Type in search box → list filters in real-time
- Empty search → shows all (respects other filters)
- Case-insensitive matching

Change 3: Delete Functionality

Request: *"Add a delete button to each task card"*

Files Modified: `App.tsx` → `TaskList.tsx` → `TaskCard.tsx`

Delete — Step 1: Lift State

```
// src/App.tsx
const [tasks, setTasks] = useState<Task[]>(sampleTasks);
```

- Tasks now in React state (not static import)
- Enables mutations (add, delete, update)

Delete — Step 2: Handler

```
// src/App.tsx
const handleDeleteTask = (taskId: string) => {
  setTasks(prev => prev.filter(t => t.id !== taskId));
};
```

- **Immutable update** — `filter` returns new array
- **Functional update** — `prev =>` ensures latest state

Delete — Step 3: Prop Drilling

```
// App.tsx
<TaskList tasks={tasks} onDeleteTask={handleDeleteTask} />

// TaskList.tsx – update interface
interface TaskListProps {
  tasks: Task[];
  onDeleteTask?: (taskId: string) => void;
}

// TaskList.tsx – pass to TaskCard
<TaskCard {...task} onDelete={() => onDeleteTask?.(task.id)} />
```

Delete — Step 4: TaskCard Button

```
// TaskCard.tsx – update interface
interface TaskCardProps {
  // ... existing
  onDelete?: () => void;
}

// In JSX footer
{onDelete && (
  <button onClick={onDelete} className="delete-btn">
    Delete
  </button>
)}
```

Delete — Verification

- Click delete → task removed from list
- No page refresh needed
- TypeScript enforces prop types

Change 4: Add Blocked Status

Request: *"Add a 'blocked' status with full filter support"*

Files Modified: `task.ts` → `ThemeContext.tsx` → `TaskCard.tsx` → `TaskList.tsx`
→ `useTaskFilter.ts`

Blocked — Step 1: Types

```
// src/types/task.ts
export type TaskStatus = 'todo' | 'in-progress' | 'done' | 'blocked';

// FilterStatus auto-includes 'blocked' via: 'all' | TaskStatus
```

Blocked — Step 2: Theme Colors

```
// src/context/ThemeContext.tsx

// In ThemeColors interface:
statusBlocked: string;

// In lightColors:
statusBlocked: '#9b59b6',

// In darkColors:
statusBlocked: '#a855f7',
```


Blocked — Step 3: TaskCard Mappings

```
// src/components/TaskCard/TaskCard.tsx

// In statusLabels:
'blocked': 'Blocked',

// In getStatusColor switch:
case 'blocked':
  return colors.statusBlocked;
```

Blocked — Step 4: TaskList & Hook

```
// TaskList.tsx – statusLabels  
'blocked': 'Blocked',  
  
// useTaskFilter.ts – statusOptions  
const statusOptions: FilterStatus[] = [  
  'all', 'todo', 'in-progress', 'done', 'blocked'  
];
```

Button appears automatically — mapped from `statusOptions` !

Blocked — Step 5: Sample Data

```
// src/data/sampleTasks.ts
{
  id: '9',
  title: 'Blocked Task Example',
  description: 'Waiting on external dependency',
  status: 'blocked',
  priority: 'high',
},
```

Blocked — Verification

- New "Blocked" filter button appears
- Cards show purple status badge
- Filtering works correctly
- TypeScript catches missing mappings

Change 5: Add Task Form

Request: *"Add a form to create new tasks"*

Files Modified: TaskForm.tsx (new) → App.tsx

Add Task — Step 1: Form Component

```
// src/components/TaskForm/TaskForm.tsx
interface TaskFormProps {
  onAddTask: (task: Omit<Task, 'id'>) => void;
}

export const TaskForm: React.FC<TaskFormProps> = ({ onAddTask }) => {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [status, setStatus] = useState<TaskStatus>('todo');
  const [priority, setPriority] = useState<TaskPriority>('medium');
  // ...
};
```

Add Task — Step 2: Form JSX

```
<form onSubmit={handleSubmit} className="task-form">
  <input
    type="text"
    placeholder="Task title"
    value={title}
    onChange={(e) => setTitle(e.target.value)}
    required
  />
  <textarea
    placeholder="Description"
    value={description}
    onChange={(e) => setDescription(e.target.value)}
  />
  {/* Status and Priority selects */}
  <button type="submit">Add Task</button>
</form>
```

Add Task — Step 3: Submit Handler

```
const handleSubmit = (e: React.FormEvent) => {  
  e.preventDefault();  
  if (!title.trim()) return;  
  
  onAddTask({ title, description, status, priority });  
  
  // Reset form  
  setTitle('');  
  setDescription('');  
  setStatus('todo');  
  setPriority('medium');  
};
```


Add Task — Step 4: App Integration

```
// src/App.tsx
const handleAddTask = (newTask: Omit<Task, 'id'>) => {
  setTasks(prev => [
    ...prev,
    { ...newTask, id: String(Date.now()) }
  ]);
};

// In JSX
<TaskForm onAddTask={handleAddTask} />
<TaskList tasks={tasks} onDeleteTask={handleDeleteTask} />
```

Add Task — Verification

- Form submits new task
- Task appears in list immediately
- Form resets after submit
- Validation prevents empty titles

Summary

Scenario	Key Concepts
dueDate	Optional props, conditional render
Search	Hook state, controlled inputs
Delete	State lifting, callback props
Blocked	Union types, Record mappings
Add Task	Form handling, Omit<T, K>

